

Atividade DWIII

Aluno: Diego da Silva Santos
GitHub: github.com/diego26-dot

Aluno: Gabriel Lima
GitHub: github.com/Ranskyth

Sumario

Atividade DWIII	1
Sumario	3
1. Objetivo da Atividade	4
2. Estrutura do Projeto	4
3. Estrutura das Rotas	5
4. Funcionamento da Aplicação	5
Arquitetura Geral	5
Fluxo de Execução	6
5. Código-Fonte Completo	6
5.1 app.js - Ponto de Entrada	6
5.2 config.js - Configuração	6
5.3 src/rotas.js - Roteador	7
5.4 lib/enviar-arquivos.js - Utilitario de Arquivos	9
5.5 index.html - Pagina Inicial	10
5.6 pages/gabriel.html	11
5.7 pages/diego.html	12
5.8 pages/gabrielSobre.html	13
5.9 pages/diegoSobre.html	16
6. Explicação dos Principais Trechos	18
6.1 Criação do Servidor HTTP	18
6.2 Parse da URL	18
6.3 Roteamento Condicional	18
6.4 Função Utilitária de Arquivos	19
6.5 Tratamento de Erro 404	20
7. Justificativa das Decisões	20
7.1 Sem Frameworks	20
7.2 Arquitetura Modular	21
7.3 Leitura Síncrona	21
7.4 CommonJS	21
8. Como Executar	21
9. Considerações Finais	22

1. Objetivo da Atividade

Nosso objetivo foi desenvolver um servidor HTTP funcional utilizando exclusivamente os módulos nativos do Node.js, **sem o uso de frameworks ou bibliotecas externas**, aplicando os conceitos fundamentais de programação web backend aprendidos na disciplina de DWIII - Desenvolvimento Web III.

A aplicação serviu como exercício prático para compreendermos como funcionam, por baixo dos panos, servidores web, roteamento de requisições, envio de arquivos estáticos e tratamento de erros HTTP.

2. Estrutura do Projeto

Nosso projeto foi organizado da seguinte forma:

```
DWIII/
|-- app.js           # Ponto de entrada do servidor
|-- config.js        # Configuracao (porta)
|-- index.html       # Pagina inicial (raiz)
|-- README.md        # Instrucoes de execucao
|
|-- src/
|   |-- rotas.js      # Roteador de URLs
|
|-- lib/
|   |-- enviar-arquivos.js # Funcao utilitaria para servir arquivos
|
|-- pages/
|   |-- gabriel.html   # Pagina do Gabriel
|   |-- gabrielSobre.html # Sobre o Gabriel
|   |-- diego.html     # Pagina do Diego
|   |-- diegoSobre.html # Sobre o Diego
|
|-- docs/
|   |-- Diego.pdf      # Curriculo do Diego
|   |-- GabrielLima.pdf # Curriculo do Gabriel
|
|-- projeto.pdf       # Descricao da atividade
```

3. Estrutura das Rotas

Rota	Arquivo Servido	Tipo	Status
/	index.html	text/html	200 ok
/gabriel	pages/gabriel.html	text/html	200 ok
/gabriel/sobre	pages/gabrielSobre.html	text/html	200 ok
/gabriel/curriculo	pages/documentos/GabrielLima.pdf	application/pdf	200 ok
/diego	pages/diego.html	text/html	200 ok
/diego/sobre	pages/diegoSobre.html	text/html	200 ok
/diego/curriculo	pages/documentos/meu-pdf.pdf	application/pdf	200 ok
/projeto	pages/documentos/projeto.pdf	application/pdf	200 ok
* (qualquer outra)	N/A	text/plain	Erro 404

4. Funcionamento da Aplicação

Arquitetura Geral

```

app.js (Ponto de Entrada)
|
|-- importa --> config.js (Configuracao: PORT)
|-- importa --> src/rotas.js (Roteador: URL -> arquivo)
|
|   |-- importa --> lib/enviar-arquivos.js (Utilitario)
|   |
|   |   |-- usa --> Node.js fs (sistema de
arquivos)

```

Fluxo de Execução

1. **Inicialização:** O `app.js` cria um servidor HTTP usando o módulo nativo `http`, vinculando a função `rotas` como handler de requisições.
2. **Requisição HTTP:** Quando um cliente envia uma requisição, a função `rotas` é chamada.
3. **Parse da URL:** A função extrai o `pathname` usando `url.parse(req.url).pathname`.
4. **Roteamento:** Um encadeamento `if/else if` compara a rota e determina qual arquivo servir.
5. **Leitura do Arquivo:** A função `enviarArquivos` lê o arquivo com `fs.readFileSync()`.
6. **Resposta:** O cabeçalho HTTP (status 200, Content-Type) é enviado, seguido do conteúdo do arquivo.
7. **Erro:** Se a leitura falhar, retorna HTTP 500 com mensagem de erro.

5. Código-Fonte Completo

5.1 `app.js` - Ponto de Entrada

```

const http = require("http");

const app = require("../src/rotas.js");

const { PORT } = require("../config.js");

let server = http.createServer(app.rotas);

server.listen(PORT);

console.log(`server on - http://localhost:${PORT}`);

```

5.2 `config.js` - Configuração

```

const PORT = 3333

```

```
module.exports = {
```

```
  PORT
```

```
}
```

5.3 src/rotas.js - Roteador

```
const url = require("url")

const enviarArquivos = require("../lib/enviar-arquivos")

const app = {

  rotas: (req, res) => {

    let rota = url.parse(req.url).pathname

    if (rota === "/" ) {

      enviarArquivos(res, "index.html", "text/html")

    }

    else if (rota === "/gabriel") {

      enviarArquivos(res, "pages/gabriel.html", "text/html")

    }

    else if (rota === "/diego") {

      enviarArquivos(res, "pages/diego.html", "text/html")

    }

    else if (rota === "/diego/curriculo") {

      enviarArquivos(res, "docs/Diego.pdf", "application/pdf")

    }

    else if(rota === "/diego/sobre"){

      enviarArquivos(res, "pages/diegoSobre.html", "text/html")

    }

  }

}
```



```
    else if (rota === "/gabriel/sobre") {  
        enviarArquivos(res, "pages/gabrielSobre.html", "text/html")  
    }  
  
    else if(rota === "/gabriel/curriculo"){  
        enviarArquivos(res, "docs/GabrielLima.pdf",  
"application/pdf")  
    }  
  
    else if(rota === "/projeto"){  
        enviarArquivos(res, "docs/projeto.pdf", "application/pdf")  
    }  
  
    else {  
        res.writeHead(404, { "Content-Type": "text/plain" })  
        res.end("error 404")  
    }  
}  
}  
}  
  
module.exports = app
```

5.4 lib/enviar-arquivos.js - Utilitário de Arquivos

```
const fs = require("fs")

const enviarArquivos = (res, arquivo, fileType) => {

  try {

    const _arquivo = fs.readFileSync(arquivo)

    res.writeHead(200, { "Content-Type": fileType })

    res.end(_arquivo)

  } catch (erro) {

    res.writeHead(500, { "Content-Type": "text/plain" })

    res.end("Erro ao carregar a página")

    console.log(erro)

  }

}

module.exports = enviarArquivos
```

5.5 index.html - Pagina Inicial

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>HomePage</title>

</head>

<body>

  <h1>Diego e Gabriel</h1>

  <h3>Este é o trabalho do Diego e do Gabriel :)</h3>

  <h3>clique nos links abaixo para acessar os projetos</h3>

  <a href="/gabriel">Gabriel</a>

  <br>

  <a href="/diego">Diego</a>

</body>

</html>
```

5.6 pages/gabriel.html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width='device-width',
initial-scale=1.0">

  <title>Gabriel</title>

</head>

<body>

  <h1>Olá, tudo bem? eu sou o gabriel! bem vindo à minha pagina
:)</h1>

  <a href="/gabriel/curriculo" target="_blank">Curriculo</a>

  <span> - </span>

  <a href="https://portfoliogabriellima.vercel.app/"
target="_blank">Portfolio</a>

  <span> - </span>

  <a href="/gabriel/sobre">Sobre</a>

</body>

</html>
```

5.7 pages/diego.html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width,
initial-scale=1.0">

  <title>Diego</title>

</head>

<body>

  <h1>Oi, eu sou dou Diego da Silva, aluno do terceiro semestre da
fatec zona sul!</h1>

  <a href="/diego/curriculo" target="_blank">

    Clique aqui para visualizar o PDF

  </a>

  <span> - </span>

  <a href="/diego/sobre">sobre</a>

</body>

</html>
```

5.8 pages/gabrielSobre.html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width,
initial-scale=1.0">

  <title>Document</title>

</head>

<body>

  <h1>Sobre o Gabriel</h1>

  <h2>Gabriel Lima</h2>

  <span>Sou um desenvolvedor fullstack, com paixão por transformar
desafios complexos em soluções elegantes e

  escaláveis. Meu trabalho abrange desenvolvimento web, APIs,
sistemas backend e integração de tecnologias

  modernas.</span>

  <h2>Experiência Profissional</h2>

  <span><strong>OAK Cell</strong>

  <br>

  <br>

  <span><strong>2021 - 2024</strong></span>

  <br>

  <br>

  Técnico de TI
```

Atendimento ao cliente e suporte técnico para resolução de problemas em computadores, notebooks e periféricos.

Configuração de roteadores, câmeras IP e impressoras de rede para ambientes corporativos.

Manutenção preventiva e corretiva de hardware, instalação de sistemas operacionais e softwares

essenciais.

<h2>Educação</h2>

FATEC - Faculdade de Tecnologia

Curso Superior de Tecnologia - Desenvolvimento de Software Multiplataforma (2025 - 2028)

ETEC - Escola Técnica Estadual de São Paulo

Técnico em Desenvolvimento de Sistemas (2024 - 2025)

<h2>Habilidades Técnicas</h2>

<h3>Backend</h3>

Node.js

Nest.js

Express

Fastify

```
<li>Chi</li>

<li>Elysia</li>

<li>Spring Boot</li>

<li>Bun</li>

</ul>


<h3>Frontend</h3>

<ul>

<li>React</li>

<li>Next.js</li>

<li>TailwindCss</li>

</ul>


<h3>DevOps</h3>

<ul>

<li> Docker </li>

<li> Kubernetes </li>

<li> Git </li>

<li> GitHub Actions </li>

</ul>


</body>

</html>
```


5.9 pages/diegoSobre.html

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width,
initial-scale=1.0">

    <title>Document</title>

</head>

<body>

<h1>Sobre o Diego</h1>

    <h2>Diego da Silva Santos</h2>

    <span> Olá, sou o Diego, aluno da FATEC Zona Sul e profissional de
TI</span>

    <h2>Habilidades Técnicas</h2>

<ul>

    <li>Python</li>

    <li>Javascript</li>

    <li>C++</li>

    <li>Php</li>

    <li>Java</li>

    <li>Pacote Office</li>

    <li>Ferramentas SQL</li>


```

```
</ul>  
</body>  
</html>
```

6. Explicação dos Principais Trechos

6.1 Criação do Servidor HTTP

```
const http = require("http");  
let server = http.createServer(app.rotas);  
server.listen(PORT);
```

O módulo nativo `http` do Node.js é utilizado para criar o servidor. A função `CreateServer` recebe a função `rotas` como callback, que será executada a cada requisição recebida. O método `listen` coloca o servidor para escutar na porta 3333.

6.2 Parse da URL

```
let rota = url.parse(req.url).pathname
```

O módulo `url` extrai apenas o patchname da requisição, ignorando query strings e fragmentos. Exemplo: `/Gabriel?foo=bar` se torna apenas `/gabriel`.

6.3 Roteamento Condicional

```
if (rota === "/") {  
    enviarArquivos(res, "index.html", "text/html")  
}  
  
else if (rota === "/gabriel") {  
    enviarArquivos(res, "pages/gabriel.html", "text/html")  
}  
  
else if (rota === "/diego") {  
    enviarArquivos(res, "pages/diego.html", "text/html")  
}  
  
else if (rota === "/diego/curriculo") {  
    enviarArquivos(res, "docs/Diego.pdf", "application/pdf")  
}
```

```

    }

    else if(rota === "/diego/sobre"){

        enviarArquivos(res, "pages/diegoSobre.html", "text/html")

    }

    else if (rota === "/gabriel/sobre") {

        enviarArquivos(res, "pages/gabrielSobre.html", "text/html")

    }

    else if(rota === "/gabriel/curriculo"){

        enviarArquivos(res, "docs/GabrielLima.pdf",
"application/pdf")

    }

    else if(rota === "/projeto"){

        enviarArquivos(res, "docs/projeto.pdf", "application/pdf")

    }

    else {

        res.writeHead(404, { "Content-Type": "text/plain" })

        res.end("error 404")

    }
}

```

O roteamento é implementado com uma cadeia de condicionais `if/else if`. Cada rota é mapeada para um arquivo e tipo MIME específico. Rotas não encontradas retornam erro 404.

6.4 Função Utilitária de Arquivos

```

const enviarArquivos = (res, arquivo, fileType) => {

    try {

```

```

    const _arquivo = fs.readFileSync(arquivo)

    res.writeHead(200, { "Content-Type": fileType })

    res.end(_arquivo)
  } catch (erro) {

    res.writeHead(500, { "Content-Type": "text/plain" })

    res.end("Erro ao carregar a página")

    console.log(erro)
  }
}

```

Função reutilizável que lê um arquivo do disco e o envia como resposta HTTP. Utiliza `fs.readFileSync` para leitura síncrona e bloco `try/catch` para tratamento de erros.

6.5 Tratamento de Erro 404

```

else {

  res.writeHead(404, { "Content-Type": "text/plain" })

  res.end("error 404")

}

```

Quando nenhuma rota é encontrada, o servidor retorna um erro HTTP 404 (Not Found) com mensagem em texto plano.

7. Justificativa das Decisões

7.1 Sem Frameworks

A decisão de não usar Express, Koa ou Satisfy foi intencional para compreendermos os fundamentos de um servidor HTTP desde a criação do socket até o envio da resposta. Isso é essencial para a disciplina de DWIII.

7.2 Arquitetura Modular

Nosso projeto foi dividido em quatro arquivos com responsabilidades claras:

- **app.js**: Criação e inicialização do servidor.
- **config.js**: Centralização de configurações.
- **src/rotas.js**: Roteamento de URLs.
- **lib/enviar-arquivos.js**: Função utilitária reutilizável.

Essa separação segue o princípio de **Responsabilidade Única (SRP)**.

7.3 Leitura Síncrona

O uso de `fs.readFileSync` simplifica o código para o contexto acadêmico. Em produção com alto tráfego, a leitura assíncrona (`fs.readFile`) seria preferível para não bloquear o event loop.

7.4 CommonJS

O projeto utiliza `require` e `module.exports` (CommonJS), o sistema de módulos nativo do Node.js, mais intuitivo para iniciantes e compatível com todas as versões.

8. Como Executar

1. Clone o repositório:
`git clone https://github.com/Ranskyth/DWIII.git`
2. Acesse a pasta do projeto:
`cd DWIII`
3. Inicie o servidor:
`node app.js`
4. Acesse no navegador:
`http://localhost:3333`

Requisito: Ter o Node.js instalado na máquina.

9. Considerações Finais

A atividade atendeu ao objetivo proposto, demonstrando nossa compreensão dos fundamentos de desenvolvimento web backend com Node.js, incluindo:

- Criação de servidores HTTP com o módulo nativo `http`.
- Roteamento manual de requisições via condicionais.
- Serviço de arquivos estáticos (HTML e PDF).
- Tratamento de erros HTTP (404 e 500).
- Organização modular do código-fonte.

Nosso projeto utiliza zero dependências externas, demonstrando nosso domínio dos módulos nativos do Node.js e uma compreensão sólida dos conceitos de programação web.